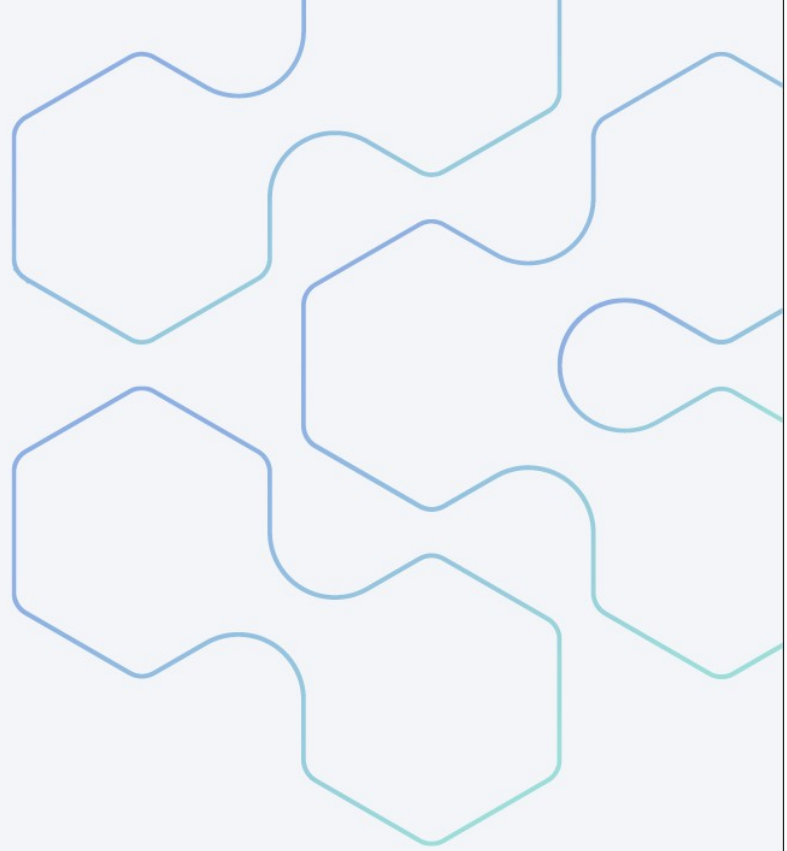




Cognition Cryptographic inventory and post-quantum cryptography readiness assessment

STIG Manager — CDRL A009 — recommendations only, pending Government authorization
Prepared by Cognition AI, Inc. · Cryptographic Modernization Specialist

2026-09-14

Table of Contents

1. Executive summary	4
2. System and boundary description	5
2.1 What the system is	5
2.2 Data and trust relationships that involve cryptography	5
2.3 Scope	5
3. Methodology	6
3.1 Known limits of the method (also in the Method sheet)	7
4. Cryptographic inventory summary	8
4.1 Top 10 rows by migration priority	9
4.2 Quantum-vulnerable (Shor) rows	10
4.3 Deprecated or weak regardless of PQC	11
4.4 Symmetric and hash rows (Grover-affected)	12
4.5 Libraries	13
4.6 TLS and certificates	14
4.7 Runtime pins	17
4.8 Secrets (type and location only)	17
5. Quantum-exposure analysis	18
5.1 What a CRQC would break here	18
5.2 Harvest-now-decrypt-later versus signature forgery	18
5.3 What is not exposed	19
6. Crypto-agility assessment	20
7. Interoperability and dependency constraints	22
7.1 Identity provider	24
7.2 JWT library	24
7.3 MySQL server	25
7.4 Node.js and OpenSSL	25
7.5 Browser	25
7.6 Mutable runtime tags	25
8. Phased PQC migration roadmap	25
Phase 0 — Inventory and governance (now)	25
Phase 1 — Hygiene (no PQC yet)	26
Phase 2 — Hybrid key establishment at TLS termination (non-production)	26
Phase 3 — Signature migration (when IdP and libraries support ML-DSA)	26
Phase 4 — Validation and retirement of classical-only paths	27
9. Risks, interoperability constraints, operational impacts	27
10. Decisions requiring Government approval	27
11. Traceability appendix	28
11.1 Recommendation → source → evidence	28
11.2 Sources (each URL was opened and confirmed during this assessment)	29
11.3 Inventory row → evidence	30

About Cognition

Cognition AI builds production-grade autonomous software engineering systems used by developers across the U.S. Government and the defense industrial base. The platform is security-first by design: the FedRAMP High authorized boundary is Devin Desktop (DoD IL4, IL5, and IL6).

At the core is Devin: cloud-based autonomous software engineering that plans and executes complex, multi-step engineering work across real codebases — autonomous, parallel, and effectively infinite in scale — in production with customers today.

Devin anchors the broader Cognition platform — DeepWiki for codebase intelligence, CLI and IDE developer tools, and Devin Review for lifecycle, CI, and change management — combined with federal audit and compliance automation (NIST, STIG) and Devin Security Swarm: complex, multi-threaded security analysis and remediation built for the hardest problems in the federal market.

1. Executive summary

STIG Manager is a web application that stores STIG evaluation results and POA&M-related data. It does not implement its own encryption of stored data. Its cryptography is concentrated in three places: (1) verification of OIDC access tokens signed by an external identity provider (IdP), (2) TLS for the API listener, the MySQL connection and the IdP connection, and (3) release signing in CI. Everything else is SHA-256 hashing for identifiers and content digests, and CSPRNG use in the browser.

The scanner produced **144 inventory rows** from **364 files** using **64 detection rules**.

Quantum-vulnerability class	Rows
Quantum-vulnerable (Shor)	17
Deprecated/weak regardless of PQC	5
Symmetric — Grover-affected, adequate at ≥ 256 -bit	6
Protocol/configuration — inherits class of negotiated algorithms	35
Not applicable (no quantum-relevant primitive)	81

Key conclusions:

- **Quantum exposure is dominated by signature forgery, not by harvest-now-decrypt-later (HNDL).** The system's only long-lived public-key trust relationships are the IdP's JWS signing key (RSA or ECDSA today) and the release-signing keys (OpenPGP in CI; ECDSA in `root.json`). A cryptographically relevant quantum computer (CRQC) would allow an attacker to forge access tokens or release signatures. It would not retroactively expose stored data, because the application stores nothing encrypted and its data has limited confidentiality life.
- **HNDL exposure exists only at the TLS layer** and only for traffic recorded today: API sessions (bearer tokens, STIG results), MySQL traffic, and IdP traffic. Bearer tokens are short-lived. STIG results are sensitive but are not secret keys; their confidentiality value decays as systems are re-scanned. This is a real but bounded exposure. It is addressed by hybrid key establishment in TLS (Phase 2), which is a deployment-layer change.
- **11 Shor-vulnerable rows are outside test code**, all of them in the JWT verification path, CI release signing and the `root.json` trust metadata. 6 further rows are test fixtures.
- **5 rows are weak regardless of PQC:** RSA-1024 keys and certificates. All are in test fixtures (`test/utls/mock0idc.js`, `test/api/mock-keycloak*`). None are in the deployed system. They should still be replaced because Node.js 24 with OpenSSL 3.5 at security level 2 rejects RSA keys under 2048 bits in TLS, and because they are a poor model of production.
- **The application never selects a signature or key-exchange algorithm itself.** Accepted JWS algorithms follow the key type published by the IdP (`jsonwebtoken 9.x` behavior) and the JWK key-type allowlist `RSA/EC/OKP` in `api/source/utls/jwksCache.js`. TLS versions and cipher suites are Node.js/OpenSSL defaults; the repository sets none. This makes the system **crypto-agile at the protocol layer but library-bound for signatures**: PQC signatures require (a) the IdP to issue ML-DSA-signed tokens, (b) a JWT library that accepts them, and (c) a one-line change to the JWK key-type filter. None of these exist in released form today.
- **The runtime is a mutable tag.** Dockerfile uses `node:lts-alpine` and CI uses `lts/*`. As of the generation date the LTS line is Node.js 24, which ships OpenSSL 3.5 (ML-KEM, ML-DSA and SLH-DSA are implemented in the library). On 2026-10-28 the `lts` tag is scheduled to move to Node.js 26. The Government should decide whether to pin a concrete version before any PQC evaluation, so results are reproducible.

Recommended immediate actions (Phase 0/1, no production change): adopt this inventory as the baseline, pin the evaluation runtime, add explicit TLS policy configuration to the deployment documentation, replace the RSA-1024 test fixtures, and open a tracking item for IdP and `jsonwebtoken` ML-DSA support. Decisions requiring Government approval are listed in Section 10.

2. System and boundary description

2.1 What the system is

- **API** (`api/source/`): Node.js/Express service. It exposes a REST API and a WebSocket log stream. It authenticates every request with an OIDC bearer token (JWT) verified against the IdP's JWKS (`api/source/utils/auth.js`, `api/source/utils/jwksCache.js`).
- **Client** (`client/src/`): browser application. It performs the OIDC authorization-code flow with PKCE in a web worker (`client/src/js/workers/oidc-worker.js`) and computes SHA-256 digests of uploaded attachments (`client/src/js/SM/Attachments.js`).
- **Database**: MySQL, reached through `mysql2`. TLS to MySQL is optional and enabled by three environment variables (`api/source/service/utils.js`).
- **Identity provider**: external. The API only consumes its discovery document and JWKS. The repository ships a test double under `test/api/mock-keycloak*` and `test/utils/mockOidc.js`.
- **Deployment**: Dockerfile (base `node:lts-alpine`), packaged binaries (`api/pkg.config.json`, Node 24 targets), GitHub Actions workflows, and a signed-image trust root (`root.json`).

2.2 Data and trust relationships that involve cryptography

Flow	Direction	Cryptography	Who controls the algorithm
Browser → IdP	login, token refresh	TLS (browser and IdP); PKCE S256 (SHA-256)	Browser, IdP
Browser → API	bearer token, REST, WebSocket	TLS if <code>STIGMAN_API_TLS_*</code> set, otherwise plaintext to a reverse proxy	Operator (certificate), Node.js defaults (protocol/ciphers)
API → IdP	discovery, JWKS fetch	TLS or HTTP depending on <code>STIGMAN_OIDC_PROVIDER</code> scheme; optional custom CA	Operator, IdP
API (verify)	JWS signature check	RS/PS/ES* per key type; RSA/EC/OKP JWKS accepted	IdP (key type), <code>jsonwebtoken</code> (algorithm set)
API → MySQL	queries	TLS if <code>STIGMAN_DB_TLS_*</code> set; <code>mysql2</code> over Node <code>tls</code>	Operator, MySQL server, Node.js defaults
CI → release	artifact signing	OpenPGP detached signatures with <code>STIGMAN_PRIVATE_KEY</code>	CI secret owner (key algorithm not visible in repo)
Image consumers	image trust	ECDSA P-256 keys in <code>root.json</code>	Repository maintainers, registry Notary service

2.3 Scope

Scanned: every text file under the repository at commit `e426316e995d04bff8ea7cdeb6803e5abce6dec1` (source tree state at scan time: clean: scanned source equals HEAD) except the exclusions below, plus `package.json/package-lock.json` for versions and JWKS/PEM material for certificate parsing. The generated artifacts are committed on top of this source, so the commit that adds them is the child of the SHA recorded here.

Not scanned or out of scope:

- `node_modules`, `dist`, `client/src/ext` (vendored ExtJS), minified bundles.

- XCCDF/CKL STIG content fixtures (`test/api/form-data-files, *.xml, *.ckl`). These describe cryptography required of the assessed systems, not cryptography used by STIG Manager.
- The scanner and its own output directory.
- The IdP, MySQL server, reverse proxy/ingress, browsers, and the container base image contents. The scanner records what the repository configures for them and flags what it does not configure. Their actual algorithms are deployment facts that must be captured in the system's deployment inventory.
- Runtime negotiation. No TLS handshake or token exchange was observed. All findings are static.

3. Methodology

`scripts/crypto_inventory.py` is a static scanner with no runtime dependencies beyond the Python standard library. `openpyxl` is required for the workbook; `cryptography` or the `openssl` CLI is used when present for certificate parsing.

What it does:

- Regex scan of text files (extensions: `.bat, .cjs, .cnf, .conf, .csv, .env, .example, .html, .ini, .js, .json, .md, .mjs, .properties, .py, .rst, .sh, .sql, .toml, .ts, .txt, .yaml, .yml`; plus `Dockerfile, .gitignore`) for asymmetric, symmetric, hash, KDF, TLS, JWT/JWS/JWKS, randomness and secret patterns.
- Node.js crypto API calls are matched by name: `generateKeyPair/Sync` (`rsa, rsa-pss, ec, ed25519, ed448, x25519, x448, dsa, dh`), `createECDH`, `diffieHellman`, `computeSecret`, `createDiffieHellman/Group`, `getDiffieHellman`, `createSign/createVerify`, `crypto.sign/verify`, `createCipheriv/createDecipheriv`, and Web Crypto `subtle.*` calls with a public-key algorithm name. Curve, prime length and cipher name are taken from the literal arguments of the same call when present. Call rules are matched against the call's first line plus the next 3 lines, so an argument list that continues on the following line is still inventoried once, at the line where the call starts.
- Password hashing and KDF calls are matched as package calls or methods (`bcrypt/bcryptjs/argon2` followed by any method such as `hash, hashSync, compare, verify`, including optional chaining), as node:crypto functions (`pbkdf2/Sync, scrypt/Sync, hkdf/Sync`) and as Web Crypto `deriveBits/deriveKey/importKey` with `PBKDF2` or `HKDF`. Aliased imports (`const h = bcrypt.hash, import { hash } from 'bcrypt'`) are not followed.
- Secret literals are matched as quoted assignments in any text file and as unquoted scalars only in `dotenv`, `shell`, `YAML`, `properties`, `ini`, `conf`, `toml` and `Dockerfile` inputs, where an unquoted value is a literal rather than an identifier; values starting with `$, %, {, *, &, <` or `(` are treated as references or placeholders.
- Literal values decide the weak class, not only the option or API name: `minVersion/maxVersion/secureProtocol` below TLS 1.2 (`SSLv2, SSLv3, TLSv1, TLSv1.1` and the `*_method` constants), cipher lists that enable `RC4, DES/3DES, NULL, export` or `MD5` suites, `createSign/createVerify/crypto.sign/verify` and Web Crypto `hash` : parameters naming `MD5` or `SHA-1`, `createHmac('md5')`, and `createCipheriv` names for `DES, RC2, RC4, Blowfish, IDEA, SEED` or `CAST` are classified `Deprecated/weak` regardless of PQC at priority 1. `rejectUnauthorized: false` keeps the protocol class but is raised to priority 1 with a stated gap. `HMAC-SHA-1` stays in the Grover class (acceptable under SP 800-131A Rev. 2). JWK fields are matched in both JavaScript (`d: '...'`) and JSON (`"d": "..."`) spelling.
- Context window of ± 12 lines is inspected to infer key sizes (`modulusLength, JWK n length`), purpose (`PKCE, kid derivation, attachment metadata`), and presence/absence of TLS or JWT verification options.
- Certificate and key files by extension (`.cer, .crt, .csr, .der, .jks, .key, .p12, .pem, .pfx`) are parsed with `cryptography` or the `openssl` CLI. PEM certificates give subject/issuer attribute types, key algorithm/size, signature algorithm and validity; PEM public keys and unencrypted private keys give key algorithm and size only (public parameters; private components are never read into the output). Files without a PEM block are tried as DER certificate, then DER private key, then DER public key, and get a `KEYMAT-DER-*` inventory row; keystores and files that parse as none of these get a `KEYMAT-FILE-UNPARSED` row (priority 2) and are recorded as not parsed. Encrypted keys and unreadable files are recorded as not parsed.

- PEM blocks (certificate, public key, private key) found in any scanned text file are parsed the same way and the parsed algorithm, size, signature algorithm and validity are copied into the KEYMAT-PEM-* inventory row; RSA/DSA below 2048 bits and SHA-1/MD5 certificate signatures are classified as deprecated/weak. Blocks that cannot be parsed keep the block-type classification and say so in the Mode column. Public keys are also parsed from JWKS x5c arrays and TUF/Notary root metadata (root.json).
- package.json and package-lock.json files are read for declared and resolved versions of libraries with a cryptographic role.
- Dockerfiles, GitHub Actions workflows and pkg build configuration are read for Node.js runtime pins.
- If node is on PATH, the local runtime is probed for Node/OpenSSL versions, default TLS versions and ML-KEM/ML-DSA availability. This describes the scanning host, not the deployed container.
- Secrets: only path, line and type are recorded; matched values are never written.
- Certificate subject/issuer: only attribute types and self-signed status are recorded unless --dn-values is given.
- Absence checks: if no HSTS header and no explicit TLS option (minVersion, maxVersion, secureProtocol, ciphers, rejectUnauthorized, secureOptions, honorCipherOrder, ecdhCurve) is found in a file of scope API (server), Client (browser) or Repository, one row per absence is added and anchored to the TLS server setup line. Matches in Documentation, Test and CI scope are inventoried but do not satisfy the check, because prose describing a control does not configure it.

Each row carries: asset ID, file:line, scope, category, algorithm, key size/curve, mode/options, purpose, library, protocol context, security relevance, quantum-vulnerability class, CNSA 2.0 target, crypto-agility rating, external dependency, migration priority (1–4) with a one-line rationale, recommended action and roadmap phase, and a redacted evidence string. Secret rows carry path, line and type only.

Classes used: Quantum-vulnerable (Shor) for RSA/ECC/DH/EdDSA; Symmetric – Grover-affected, adequate at ≥ 256 -bit for AES/SHA-2; Deprecated/weak regardless of PQC for MD5/SHA-1/3DES/RSA<2048/TLS<1.2; Protocol/configuration – inherits class of negotiated algorithms for TLS and JWT plumbing whose algorithms are chosen elsewhere; Not applicable for secrets, randomness, identifiers and runtime facts.

Priority scale: **1** act in Phase 1 or blocks later phases; **2** Phase 1/2 configuration and TLS work; **3** track, depends on an external party or is test-only; **4** informational.

3.1 Known limits of the method (also in the Method sheet)

- Static text matching only. Dynamic algorithm selection (for example the algorithm list jsonwebtoken derives from the key type) is inferred from library source knowledge, not observed at runtime.
- Key sizes are recorded only where they appear in source (modulusLength), can be derived from embedded key material (JWK n, SPKI), or are stated in comments. Keys supplied at deployment time (TLS certificates, IdP signing keys) are not visible to the scanner.
- TLS protocol versions and cipher suites negotiated at runtime depend on the Node.js/OpenSSL build of the container image and on peers (reverse proxy, MySQL server, IdP, browsers). The scanner records what the repository configures and flags what it does not configure.
- Vended third-party code (client/src/ext), minified bundles, lockfiles (except for version extraction), XCCDF/CKL STIG content fixtures (test/api/form-data-files, .xml, .ckl), generated docs, this scanner and its own output (the canonical docs/security/crypto-inventory directory, any in-repository --out directory, and the concrete JSON/XLSX/report/template output paths, so that --out . does not inventory the previous run) are not pattern-scanned.
- Text PEM files with certificate/key extensions are parsed for metadata and also run through the pattern rules, so a committed private key appears both in the certificate table and as a Secret row. A PEM block may span at most 400 lines; longer blocks are recorded as not parsed. Binary keystores (.p12, .pfx, .jks) are inventoried by path only; their contents are not parsed (password required).
- Prose in documentation is scanned only for configuration identifiers (environment variable names, TLS directives). Narrative mentions of algorithms in release notes or user guides are not inventoried.

- Secret detection uses simple assignment patterns and JWT/JWK/PEM shapes. It will miss encoded or split secrets and may flag placeholder values used in tests; each Secret row is labeled with its scope (Test, CI, Documentation).
- The `node:lts-alpine` tag and `lts/*` CI alias resolve to different Node versions over time. The resolved version is recorded only when supplied with `--lts-resolves-to`.
- Occurrences of the same JWT literal pattern inside a single test file are collapsed to one row (first line, count in the rationale).
- `commit_sha` is HEAD of the checkout at scan time and `source_tree_vs_commit` records whether the scanned source matched it. Because the generated artifacts are committed on top of that source, the SHA recorded inside an artifact is always the parent of the commit that adds the artifact, never that commit itself.

4. Cryptographic inventory summary

Counts below are generated from the scanner output.

By migration priority

Migration priority	Rows
1	3
2	13
3	59
4	69

By category

Category	Rows
Asymmetric	2
Certificate/Key material	1
Hash	6
JWT/JWS/JWKS	16
Randomness	7
Runtime	9
Secret	53
Secret reference	9
Signing	9
TLS	32

By scope

Scope	Rows
API (server)	32
CI	29
Client (browser)	10
Documentation	16
Repository	6
Test	51

By crypto-agility rating

Crypto-agility rating	Rows
Configurable	55
Hardcoded	75
Library-bound	5

Crypto-agility rating	Rows
N/A	9

4.1 Top 10 rows by migration priority

Class abbreviations used in the tables below: Shor = Quantum-vulnerable (Shor); Grover = Symmetric — Grover-affected, adequate at ≥ 256 -bit; Weak = Deprecated/weak regardless of PQC; Protocol = Protocol/configuration — inherits class of negotiated algorithms; N/A = Not applicable. P = migration priority (1 highest). Rows whose location starts with `test/` are test scope.

Asset	Location	Algorithm	Class	P	Rationale
CI-0002	api/source/utils/auth.js:77	JWS signature verification (RS/PS/ES* per key type)	Shor	1	Primary authentication control of the API. Signature algorithm is set by the IdP and accepted by library default.
CI-0007	api/source/utils/jwksCache.js:188	JWK import: RSA/EC/OKP (RSA, ECDSA, EdDSA key types)	Shor	1	Every API request is authenticated with a signature verified against these keys. Key type allowlist is hardcoded.
CI-0040	api/source/utils/jwksCache.js:92	Plaintext HTTP permitted for JWKS retrieval	Protocol	1	Signing keys fetched without TLS can be substituted on path. Default authority is <code>http://localhost:8080</code> (development).
CI-0005	api/source/utils/config.js:97	Configuration flag	N/A	2	Must remain unset in production; verify in deployment configuration.
CI-0027	api/source/bootstrap/server.js:45	Explicit TLS options (none set)	Protocol	2	Absence recorded so the control can be assigned to the deployment layer or added as configuration.
CI-0028	api/source/bootstrap/server.js:45	TLS server (native HTTPS)	Protocol	2	Key exchange in TLS is the harvest-now-decrypt-later exposure point for tokens and data in transit.
CI-0029	api/source/bootstrap/server.js:48	Plaintext HTTP listener	Protocol	2	Confidentiality of bearer tokens then depends entirely on an external TLS terminator.
CI-0030	api/source/service/utils.js:198	TLS client (MySQL)	Protocol	2	STIG/POA&M data and credentials transit this

Asset	Location	Algorithm	Class	P	Rationale
					link; key exchange is the quantum-exposed step.
CI-0038	api/source/utils/config.js:95	Plaintext default for OIDC authority	Protocol	2	Default is http://; production must override with an https URL.
CI-0052	.github/workflows/build-binary-artifacts.yml:65	OpenPGP detached signature (key algorithm not visible in repository)	Shor	2	Software signing is a CNSA 2.0 early-transition category; the signing key algorithm is held outside the repository.

4.2 Quantum-vulnerable (Shor) rows

Asset	Location	Algorithm	Key size/curve	Agility	P
CI-0002	api/source/utils/auth.js:77	JWS signature verification (RS/PS/ES* per key type)		Library-bound	1
CI-0007	api/source/utils/jwksCache.js:188	JWK import: RSA/EC/OKP (RSA, ECDSA, EdDSA key types)		Hardcoded	1
CI-0012	test/utils/mockOidc.js:23	RS256		Hardcoded	3
CI-0013	test/utils/mockOidc.js:65	RS256		Hardcoded	3
CI-0014	test/utils/mockOidc.js:185	JWS signing		Hardcoded	3
CI-0015	test/utils/mockOidc.js:189	JWS signing		Hardcoded	3
CI-0016	test/utils/mockOidc.js:212	JWS signing		Hardcoded	3
CI-0051	.github/workflows/build-binary-artifacts.yml:49	OpenPGP private key import		Configurable	3
CI-0052	.github/workflows/build-binary-artifacts.yml:65	OpenPGP detached signature (key algorithm not visible in repository)		Configurable	2
CI-0053	.github/workflows/build-binary-artifacts.yml:69	OpenPGP detached signature (key algorithm not		Configurable	2

Asset	Location	Algorithm	Key size/ curve	Agility	P
		visible in repository)			
CI-0054	.github/workflows/build-binary-artifacts.yml:78	OpenPGP signature verification		Configurable	3
CI-0055	.github/workflows/build-binary-artifacts.yml:82	OpenPGP signature verification		Configurable	3
CI-0056	root.json:1	ECDSA	secp256r1	Hardcoded	2
CI-0057	root.json:1	ECDSA (X.509 certificate)	secp256r1	Hardcoded	2
CI-0058	root.json:1	ECDSA	secp256r1	Hardcoded	2
CI-0059	root.json:1	ECDSA	secp256r1	Hardcoded	2
CI-0135	test/utis/mockOidc.js:53	Private key (JWK private exponent)		Hardcoded	3

4.3 Deprecated or weak regardless of PQC

Asset	Location	Algorithm	Key size/ curve	Rationale
CI-0009	test/api/mock-keycloak-test-cases/no-jwks/auth/realms/stigman/protocol/openid-connect/certs:1	RSA (JWK), alg=RS256	RSA-1024	RSA-1024 is below the SP 800-131A Rev. 2 floor. Static test JWKS; the key is the widely published insecure default and is on the application denylist. Test scope: not deployed.
CI-0010	test/api/mock-keycloak-test-cases/secure-kid/auth/realms/stigman/protocol/openid-connect/certs:1	RSA (JWK), alg=RS256	RSA-1024	RSA-1024 is below the SP 800-131A Rev. 2 floor. Static test JWKS; the key is the widely published insecure default and is on the application denylist. Test scope: not deployed.
CI-0011	test/api/mock-keycloak/auth/realms/stigman/protocol/openid-connect/certs:1	RSA (JWK), alg=RS256	RSA-1024	RSA-1024 is below the SP 800-131A Rev. 2 floor. Static test JWKS; the key is the widely published insecure

Asset	Location	Algorithm	Key size/ curve	Rationale
				default and is on the application denylist. Test scope: not deployed.
CI-0017	test/utls/mockOidc.js:21	RSA	RSA-1024	RSA-1024 is below the 2048-bit floor of SP 800-131A Rev. 2 and is quantum-vulnerable. Generates the signing key pair used to issue tokens; algorithm and size are literals in code. Test scope: not deployed.
CI-0018	test/utls/mockOidc.js:48	RSA	RSA-1024	Private key material is embedded in source; the key reproduces a known insecure IdP default key for negative tests. Test scope: not deployed.

4.4 Symmetric and hash rows (Grover-affected)

SHA-256 is the only symmetric-class primitive found. It is used for PKCE, content digests and key identifiers. None of these uses protects long-lived confidentiality. CNSA 2.0 lists SHA-384 and SHA-512; SHA-256 remains acceptable under SP 800-131A Rev. 2. No AES, ChaCha20, 3DES, MD5, SHA-1, bcrypt, Argon2 or PBKDF2 use was found in application code. The application does not hash passwords; authentication is delegated to the IdP.

Asset	Location	Algorithm	Purpose	Security-relevant
CI-0060	api/source/service/STIGService.js:799	SHA-256	Content digest used to detect duplicate STIG check/fix text	No (deduplication)
CI-0061	api/source/service/STIGService.js:802	SHA-256	Content digest used to detect duplicate STIG check/fix text	No (deduplication)
CI-0062	client/src/js/SM/Attachments.js:141	SHA-256	Digest of uploaded attachment stored as metadata (client-computed, not verified server-side)	No (metadata / identification)
CI-0063	client/src/js/workers/oidc-worker.js:157	SHA-256	PKCE code challenge (S256) for the OIDC authorization code flow	Yes
CI-0064	client/src/js/workers/	SHA-256	Bind the OAuth	Yes

Asset	Location	Algorithm	Purpose	Security-relevant
	oidc-worker.js:267	(PKCE S256)	authorization code to the browser client	
CI-0065	test/utis/mockOidc.js:37	SHA-256	Derive a JWK key identifier (kid) from the DER public key	No (identifier)

4.5 Libraries

Library	Resolved version	Crypto role	PQC support today
jsonwebtoken	api/source/package-lock.json: 9.0.2; test/utis/package-lock.json: 9.0.2	JWT/JWS parse and signature verification (API access tokens)	None. Supports HS, RS, PS, ES only; no EdDSA, no ML-DSA. Accepted algorithms are chosen from the key type when algorithms is not passed.
jwt-rsa	api/source/package-lock.json: 3.2.0	JWKS client (declared dependency)	None. RSA/EC JWKS import only.
jose	api/source/package-lock.json: 4.15.9	JOSE/JWK library (transitive via jwt-rsa)	None in the resolved 4.x line.
mysql2	api/source/package-lock.json: 3.15.3	MySQL client; TLS to the database through Node tls	Inherits Node/OpenSSL TLS capabilities. Hybrid ML-KEM key exchange requires Node with OpenSSL 3.5+ and a MySQL server built against OpenSSL 3.5+.
undici	api/source/package-lock.json: 6.27.0	HTTP client for OIDC discovery; TLS to the identity provider	Inherits Node/OpenSSL TLS capabilities.
ws	api/source/package-lock.json: 8.21.0; test/api/package-	WebSocket server (log stream); relies on	Inherits Node/OpenSSL TLS

Library	Resolved version	Crypto role	PQC support today
	lock.json: 8.21.0	the HTTP(S) server TLS	capabilities.
express	api/source/package-lock.json: 4.22.2	HTTP framework; no cryptography of its own	N/A
jszip	api/source/package-lock.json: 3.10.1; test/api/package-lock.json: 3.10.1	ZIP read/write; CRC-32 only (non-cryptographic)	N/A
archiver	api/source/package-lock.json: 7.0.1	Archive creation; CRC-32 only (non-cryptographic)	N/A
node:crypto / node:tls / node:https (built-in)	Repository pins: node:its-alpine (Dockerfile); node:its-alpine (.github/workflows/api-container-tests.yml); node24-win,node24-linuxstatic (api/pkg.config.json). Scanning host: Node 24.19.0 / OpenSSL 3.5.7. Its tag resolution supplied: Node.js 24 (active LTS until 2026-10-28 per nodejs/Release schedule; bundles OpenSSL 3.5 from 24.5.0)	JWK import, RSA key generation (tests), SHA-256 digests, TLS server and clients	ML-KEM and ML-DSA APIs are available in Node.js 24.x builds with OpenSSL 3.5+ (see Runtime probe). jsonwebtoken does not use them.
Web Crypto API (browser crypto.subtle / getRandomValues)	Set by the user's browser	PKCE S256 digest, random verifier/state/nonce, attachment digest	No public-key operations performed in the client; TLS to API and IdP is provided by the browser.

4.6 TLS and certificates

Rows in the TLS category (documentation-scope rows omitted here; see the workbook):

Asset	Location	Item	Observations
CI-0019	api/launchers/stig-manager.sh:40	STIGMAN_API_TLS_CERT_FILE	Commented launcher template entry for a TLS variable
CI-0020	api/launchers/stig-manager.sh:51	STIGMAN_API_TLS_KEY_FILE	Commented launcher template entry for a TLS variable
CI-0021	api/launchers/stig-manager.sh:62	STIGMAN_API_TLS_KEY_PASSPHRASE	Commented launcher template entry for a TLS variable
CI-0022	api/launchers/stig-manager.sh:341	STIGMAN_DB_TLS_CA_FILE	Commented launcher template entry for a TLS

Asset	Location	Item	Observations
			variable
CI-0023	api/launchers/stig-manager.sh:352	STIGMAN_DB_TLS_CERT_FILE	Commented launcher template entry for a TLS variable
CI-0024	api/launchers/stig-manager.sh:364	STIGMAN_DB_TLS_KEY_FILE	Commented launcher template entry for a TLS variable
CI-0025	api/launchers/stig-manager.sh:603	STIGMAN_OIDC_CA_CERTS	Commented launcher template entry for a TLS variable
CI-0026	api/source/bootstrap/server.js:45	HSTS header (not configured)	No Strict-Transport-Security header or helmet() use in application code or deployable configuration (documentation, tests and CI excluded); browsers are not instructed to require HTTPS by the API itself.
CI-0027	api/source/bootstrap/server.js:45	Explicit TLS options (none set)	No minVersion, maxVersion, ciphers, rejectUnauthorized, secureOptions, honorCipherOrder or ecdhCurve literal in application code or deployable configuration (documentation, tests and CI excluded). Node.js defaults apply to the API server, the MySQL client and the IdP clients.
CI-0028	api/source/bootstrap/server.js:45	TLS server (native HTTPS)	Options set: key, cert, passphrase. Not set in code: minVersion, maxVersion, ciphers, secureOptions, honorCipherOrder, ecdhCurve (Node defaults apply).
CI-0029	api/source/bootstrap/server.js:48	Plaintext HTTP listener	Default listener when TLS files are not configured (TLS expected at a reverse proxy)
CI-0030	api/source/service/utls.js:198	TLS client (MySQL)	ssl object built from ca/cert/key files. Not set in code:

Asset	Location	Item	Observations
			rejectUnauthorized, minVersion, ciphers (mysql2/Node defaults apply; Node verifies the server certificate by default).
CI-0031	api/source/utls/auth.js:247	TLS client (OIDC discovery) with custom CA	Validate the IdP certificate chain using STIGMAN_OIDC_CA_CERTS
CI-0032	api/source/utls/config.js:64	STIGMAN_API_TLS_KEY_FILE	Path/passphrase configuration for native TLS
CI-0033	api/source/utls/config.js:65	STIGMAN_API_TLS_KEY_PASSPHRASE	Path/passphrase configuration for native TLS
CI-0034	api/source/utls/config.js:66	STIGMAN_API_TLS_CERT_FILE	Path/passphrase configuration for native TLS
CI-0035	api/source/utls/config.js:77	STIGMAN_DB_TLS_CA_FILE	Enable TLS (and optional client-certificate auth) to MySQL
CI-0036	api/source/utls/config.js:78	STIGMAN_DB_TLS_CERT_FILE	Enable TLS (and optional client-certificate auth) to MySQL
CI-0037	api/source/utls/config.js:79	STIGMAN_DB_TLS_KEY_FILE	Enable TLS (and optional client-certificate auth) to MySQL
CI-0038	api/source/utls/config.js:95	Plaintext default for OIDC authority	Development default for the identity provider URL
CI-0039	api/source/utls/jwksCache.js:89	TLS client (JWKS fetch) with custom CA	Validate the IdP certificate chain when fetching JWKS
CI-0040	api/source/utls/jwksCache.js:92	Plaintext HTTP permitted for JWKS retrieval	Fetch signing keys over http:// when the discovered jwks_uri is not https

Certificate material parsed from the tree (subject/issuer attribute types only; values withheld):

Location	Source	Key / signature	Valid to	Subject
test/api/mock-keycloak/auth/realms/stigman/protocol/openid-connect/certs:1	JWKS x5c[0] (kid FJ86GcF3jTbN...)	RSA-1024 / sha256WithRSAEncryption	2029-06-24	[values withheld] attributes: commonName; self-signed (subject = issuer)
test/api/mock-keycloak-test-cases/no-jwks/auth/	JWKS x5c[0] (kid	RSA-1024 / sha256WithRSAEncryption	2029-06-24	[values withheld]

Location	Source	Key / signature	Valid to	Subject
realms/stigman/protocol/openid-connect/certs:1	xxxxxxF3jTbN...)	ion		attributes: commonN ame; self-signed (subject = issuer)
test/api/mock-keycloak-test-cases/secure-kid/auth/realms/stigman/protocol/openid-connect/certs:1	JWKS x5c[0] (kid xxxxxxF3jTbN...)	RSA-1024 / sha256WithRSAEncryption	2029-06-24	[values withheld] attributes: commonN ame; self-signed (subject = issuer)

No PEM, CRT, KEY, JKS or P12 files are committed. `api/source/tls/` contains only a README that says TLS certificates and keys may be placed there; TLS material is supplied at deployment through `STIGMAN_API_TLS_KEY_FILE`, `STIGMAN_API_TLS_CERT_FILE` and `STIGMAN_DB_TLS_*`. `.gitignore` excludes `**/*.pem` and the MySQL TLS helper files under `api/source/tls/mysql*/`, which is the intended state. It does not exclude `.key`, `.crt`, `.p12` or `.jks`; adding those patterns is a Phase 1 hygiene item.

4.7 Runtime pins

Location	Source	Value
Dockerfile:15	Node.js/ OpenSSL runtime (container base image)	node:its-alpine
.github/workflows/api-container-tests.yml:40	Node.js runtime (container test matrix)	node:its-alpine
api/pkg.config.json:14	Node.js runtime (packaged binaries)	node24-win,node24-linuxstatic

Local probe of the scanning host (not the deployed image): node: 24.19.0; openssl: 3.5.7; tls_default_min: TLSv1.2; tls_default_max: TLSv1.3; ml-kem-768: available; ml-dsa-65: available. Resolution of the `lts` tag supplied at scan time: Node.js 24 (active LTS until 2026-10-28 per nodejs/Release schedule; bundles OpenSSL 3.5 from 24.5.0).

4.8 Secrets (type and location only)

53 rows record hardcoded credential-shaped values. All are in test, CI or documentation scope. No value is reproduced in any artifact. The JWT literals are test tokens signed by the repository's own RSA-1024 test key; the database passwords are those of throwaway containers in test and CI configuration and in deployment examples. They are inventoried because they show where a real credential could be pasted by mistake.

Scope	Type	Rows
CI	Database password literal (MYSQL_PASSWORD)	3

Scope	Type	Rows
CI	Database password literal (MYSQL_ROOT_PASSWORD)	3
CI	Database password literal (STIGMAN_DB_PASSWORD)	3
Documentation	Database password literal (MYSQL_PASSWORD)	2
Documentation	Database password literal (MYSQL_ROOT_PASSWORD)	2
Documentation	Database password literal (STIGMAN_DB_PASSWORD)	1
Test	Database password literal (MYSQL_PASSWORD)	2
Test	Database password literal (MYSQL_ROOT_PASSWORD)	2
Test	Database password literal (STIGMAN_DB_PASSWORD)	14
Test	JWT literal	20
Test	Private key (JWK private exponent)	1

5. Quantum-exposure analysis

5.1 What a CRQC would break here

A CRQC running Shor's algorithm breaks RSA, ECDSA, ECDH, EdDSA and finite-field DH. In this system those primitives appear in exactly four places:

- **IdP token signatures** (`api/source/utils/auth.js:77`, `api/source/utils/jwksCache.js:188`). The API trusts any token whose signature verifies under a key from the IdP's JWKS. The key is RSA or EC today. With the IdP's private key recovered, an attacker mints tokens for any user and role. This is the highest-consequence exposure and it is an *authentication/integrity* exposure.
- **TLS key exchange and server authentication** for browser→API, API→IdP and API→MySQL (`api/source/bootstrap/server.js:45`, `api/source/service/utils.js:198`, `api/source/utils/jwksCache.js:92`). Today these negotiate ECDHE (X25519 or P-256) with RSA or ECDSA certificates under Node.js defaults. Recorded traffic could be decrypted later if the ephemeral exchange is broken (HNDL). Server impersonation would also become possible, but only with a live CRQC.
- **Release signing** (`.github/workflows/build-binary-artifacts.yml:65-82`). Detached OpenPGP signatures over binaries. The key algorithm is not visible in the repository; the key lives in a CI secret. Forgery lets an attacker ship a trojaned binary that passes the published verification step.
- **Image trust metadata** (`root.json:1`). ECDSA P-256 public keys for a registry Notary trust root. Same consequence as 3, for container images.

5.2 Harvest-now-decrypt-later versus signature forgery

Exposure	Data at risk	Lifetime of the data	Attack timing	Assessment
HNDL on browser→API TLS	Bearer tokens; STIG results, asset names, POA&M text	Tokens: minutes. Results: months to years (re-scanned periodically)	Record now, decrypt after CRQC	Bounded. Results are sensitive (they list unremediated weaknesses) but are refreshed; historical results

Exposure	Data at risk	Lifetime of the data	Attack timing	Assessment
				lose value as systems change. Still worth hybrid key establishment because the fix is cheap and at the TLS terminator.
HNDL on API→MySQL TLS	Same data plus schema and credentials in flight	Same	Same	Same; also depends on the MySQL server build. Often on a private network.
HNDL on API→IdP TLS	JWKS (public), discovery (public)	Public data	n/a	Negligible. The JWKS is public by design.
Token forgery	Full application access	n/a	Requires a live CRQC	Highest consequence, but not retroactive. Mitigated when the IdP signs with ML-DSA and the API accepts it.
Release/image signature forgery	Supply chain	Signatures verified at install time	Requires a live CRQC	High consequence. Mitigated by ML-DSA or stateful hash-based signatures (LMS/XMSS) for firmware-like artifacts.

Conclusion: the exposure profile of a read-only STIG/POA&M management system is mostly **integrity and authentication**. The data it holds is compliance evidence, not long-lived secrets. The HNDL surface is real but bounded to TLS, and TLS is where hybrid PQC is available first. Signature migration matters more for this system than key-establishment migration, and signature migration depends entirely on the IdP and library ecosystem.

5.3 What is not exposed

- No data-at-rest encryption is performed by the application, so there is no application-managed key to migrate. Database and disk encryption belong to the hosting environment and must be inventoried there.
- No password hashing. Authentication is delegated.
- No application-level encryption of attachments or exports.
- SHA-256 uses (PKCE, digests, `kid`) are collision/preimage uses with no public-key component. Grover's algorithm reduces effective preimage strength to about 128 bits, which remains adequate; CNSA 2.0 prefers SHA-384 for new designs.

6. Crypto-agility assessment

Element	Where the algorithm is decided	Rating	Notes
JWS signature algorithm	IdP key type; jsonwebtoken maps key type → RS/PS/ES* (auth.js:77)	Library-bound	No algorithms option is passed. Swapping the IdP key from RSA to EC changes the accepted set with no code change. Adding ML-DSA needs a library that knows the alg and a JWK kty the filter accepts.
JWK key-type allowlist	jwtCache.js:188 — literal RSA, EC, OKP	Hardcoded	A one-line change, but a change. ML-DSA JWKs use a different kty (AKP in the current IETF draft).
Audience / issuer checks	config.js STIGMAN_JWT_AUD_VALUE (audience optional); issuer from discovery	Configurable	Not PQC-relevant; noted because audience check is off unless configured.
TLS versions and ciphers	Not in repository; Node.js/ OpenSSL defaults	Library-bound	Configurable only by changing the image or adding options in code. Default floor is TLS 1.2, ceiling TLS 1.3 on Node 24.
TLS certificate/key	STIGMAN_API_TLS_*, STIGMAN_DB_TLS_* files	Configurable	Any key type Node accepts. ML-DSA certificates would need OpenSSL 3.5+ and a CA that issues them.
Hybrid key exchange	Node.js tls groups option /	Library-bound	Node 24 with

Element	Where the algorithm is decided	Rating	Notes
(X25519MLKEM768)	OpenSSL defaults		OpenSSL 3.5 can offer it when the group is in the default list or set explicitly. Not exposed as configuration in this repository.
HSTS and security headers	Not set in application	Absent	Must be set by the reverse proxy.
Release signing	GnuPG in CI; key from secret	Configurable	Algorithm follows the key generated by the secret owner. GnuPG ML-DSA support is not assumed here.
Image trust root	root.json static ECDSA keys	Hardcoded	Rotation requires a new trust root and consumer update.

Gaps generated from the inventory:

Asset	Location	Algorithm	Rating	Gap
CI-0002	api/source/utils/auth.js:77	JWS signature verification (RS/PS/ES* per key type)	Library-bound	Accepted JWS alg values are not pinned in configuration; they follow the library default for the key type.
CI-0007	api/source/utils/jwksCache.js:188	JWK import: RSA/EC/OKP (RSA, ECDSA, EdDSA key types)	Hardcoded	Accepted JWK key types (RSA, EC, OKP) are a literal allowlist in code; ML-DSA JWKs (kty AKP) would be dropped.
CI-0026	api/source/bootstrap/server.js:45	HSTS header (not configured)	Library-bound	No Strict-Transport-Security header or helmet() use in application code or deployable configuration (documentation, tests and CI excluded); browsers are not instructed to require HTTPS by the API itself.
CI-0027	api/source/bootstrap/	Explicit TLS options (none)	Library-bound	No minVersion, maxVersion, ciphers, rejectUnauthorized,

Asset	Location	Algorithm	Rating	Gap
	server.js:45	set)		secureOptions, honorCipherOrder or ecdhCurve literal in application code or deployable configuration (documentation, tests and CI excluded). Node.js defaults apply to the API server, the MySQL client and the IdP clients.
CI-0028	api/source/bootstrap/server.js:45	TLS server (native HTTPS)	Library-bound	TLS protocol versions and cipher suites are not exposed as configuration; only key/cert/passphrase file paths are.
CI-0030	api/source/service/utls.js:198	TLS client (MySQL)	Library-bound	MySQL TLS version and cipher policy are not configurable from the application; verification behavior relies on library defaults.
CI-0056	root.json:1	ECDSA	Hardcoded	Trust root keys are static ECDSA public keys embedded in root.json.
CI-0057	root.json:1	ECDSA (X.509 certificate)	Hardcoded	Trust root keys are static ECDSA public keys embedded in root.json.
CI-0058	root.json:1	ECDSA	Hardcoded	Trust root keys are static ECDSA public keys embedded in root.json.
CI-0059	root.json:1	ECDSA	Hardcoded	Trust root keys are static ECDSA public keys embedded in root.json.

7. Interoperability and dependency constraints

Assets	Component	External party	Algorithm / item	Class
CI-0002	api/source/utls/auth.js	Identity provider signing algorithm; jsonwebtoken algorithm support	JWS signature verification (RS/PS/ES* per key type)	Shor
CI-0006	api/source/utls/config.js	Identity provider key rotation schedule	JWKS cache/rotation window	N/A
CI-0007	api/source/utls/jwksCache.js	Identity provider (OIDC) publishes	JWK import: RSA/EC/OKP (RSA, ECDSA, EdDSA key)	Shor

Assets	Component	External party	Algorithm / item	Class
		JWKS; key algorithm chosen by the IdP	types)	
CI-0026	api/source/bootstrap/server.js	Reverse proxy / ingress must set HSTS	HSTS header (not configured)	Protocol
CI-0027	api/source/bootstrap/server.js	Node.js/OpenSSL build defaults in the container image	Explicit TLS options (none set)	Protocol
CI-0028	api/source/bootstrap/server.js	Operator-supplied certificate/key; browser and reverse-proxy TLS stacks; Node/OpenSSL build in the container image	TLS server (native HTTPS)	Protocol
CI-0029	api/source/bootstrap/server.js	Reverse proxy / ingress TLS termination	Plaintext HTTP listener	Protocol
CI-0030	api/source/service/utls.js	MySQL server TLS versions and cipher suites	TLS client (MySQL)	Protocol
CI-0031, CI-0039	api/source/utls/auth.js	Identity provider TLS certificate and CA	TLS client (OIDC discovery) with custom CA	Protocol
CI-0032	api/source/utls/config.js	Operator-supplied PEM material (algorithm chosen at deployment)	STIGMAN_API_TLS_KEY_FILE	Protocol
CI-0033	api/source/utls/config.js	Operator-supplied PEM material (algorithm chosen at deployment)	STIGMAN_API_TLS_KEY_PASSPHRASE	Protocol
CI-0034	api/source/utls/config.js	Operator-supplied PEM material (algorithm chosen at deployment)	STIGMAN_API_TLS_CERT_FILE	Protocol
CI-0035	api/source/utls/config.js	MySQL server TLS configuration and OpenSSL build	STIGMAN_DB_TLS_CA_FILE	Protocol
CI-0036	api/source/utls/config.js	MySQL server TLS configuration and OpenSSL build	STIGMAN_DB_TLS_CERT_FILE	Protocol
CI-	api/source/utls/	MySQL server TLS	STIGMAN_DB_TLS_KEY_FILE	Protocol

Assets	Component	External party	Algorithm / item	Class
0037	config.js	configuration and OpenSSL build	E	ol
CI-0038	api/source/utis/config.js	Identity provider	Plaintext default for OIDC authority	Protocol
CI-0040	api/source/utis/jwksCache.js	Identity provider URL scheme (STIGMAN_OIDC_PROVIDER)	Plaintext HTTP permitted for JWKS retrieval	Protocol
CI-0041	docs/installation-and-setup/db.rst	MySQL server	MySQL server TLS enforcement (documentation)	Protocol
CI-0051	.github/workflows/build-binary-artifacts.yml	CI secret store	OpenPGP private key import	Shor
CI-0052, CI-0053	.github/workflows/build-binary-artifacts.yml	CI secret STIGMAN_PRIVATE_KEY; consumers' GnuPG versions	OpenPGP detached signature (key algorithm not visible in repository)	Shor
CI-0056, CI-0057, CI-0058, CI-0059	root.json	Container registry Notary service and consumer docker clients	ECDSA	Shor
CI-0064	client/src/js/workers/oidc-worker.js	Identity provider must support PKCE S256	SHA-256 (PKCE S256)	Grover
CI-0073	Dockerfile	Upstream Node.js image publisher (mutable tag)	Node.js/OpenSSL runtime (container base image)	Protocol

7.1 Identity provider

The API accepts whatever the IdP publishes, within the RSA/EC/OKP filter. Migration to ML-DSA-signed tokens requires the IdP to (a) generate an ML-DSA key, (b) publish it in JWKS with the right `key/alg`, and (c) sign access tokens with it. A JOSE algorithm registration for ML-DSA is in progress at the IETF (draft-ietf-cose-dilithium). Keycloak upstream has an open pull request for ML-DSA OIDC support (keycloak/keycloak#50358, open, not merged at the time of writing). No released Keycloak version is assumed to support it. Keycloak's active/passive key model allows adding a new signing key and rotating without downtime, which is the mechanism a later cutover would use.

7.2 JWT library

`jsonwebtoken` 9.0.2 supports HS, RS, PS and ES only. It does not support EdDSA, so the OKP entry in the JWK filter is currently unreachable in practice. It has no ML-DSA support. A library change (for example to a `jwt` library)

release that implements the ML-DSA draft) is a prerequisite for Phase 3 and is a code change that requires Government authorization.

7.3 MySQL server

`mysql2` uses Node's `tls`. The negotiated protocol and groups are the intersection of the Node/OpenSSL build and the MySQL server build. Hybrid ML-KEM groups require both sides on OpenSSL 3.5-class libraries. The application does not set `rejectUnauthorized`; Node's default verifies the server certificate against the supplied CA, but this should be confirmed in a test environment rather than assumed.

7.4 Node.js and OpenSSL

- Node.js 24 became LTS on 2025-10-28 and ships OpenSSL 3.5 from 24.5.0 onward (Node.js release notes). OpenSSL 3.5 implements ML-KEM, ML-DSA and SLH-DSA and is an LTS release supported to April 2030 (OpenSSL announcement).
- Node.js 24 uses OpenSSL security level 2 by default: RSA/DSA/DH keys under 2048 bits and EC keys under 224 bits are rejected in TLS (Node.js v22→v24 migration guide). This is why the RSA-1024 test fixtures are flagged even though they never touch TLS today.
- The scanning host (Node 24.19.0, OpenSSL 3.5.7) reported ML-KEM-768 and ML-DSA-65 key generation available through `node:crypto`. This is evidence for the Node 24 line, not for the deployed image, which is a mutable tag.
- Per the Node.js release schedule, the `lts` tag moves to Node.js 26 on 2026-10-28. The bundled OpenSSL of that line should be re-verified at that time.
- Packaged binaries target `node24-win` and `node24-linuxstatic` (`api/pkg.config.json:14`). Their embedded OpenSSL is whatever the packaging tool's Node 24 build bundles.

7.5 Browser

The browser negotiates TLS to the API and IdP. Chrome offers the X25519MLKEM768 hybrid group in TLS 1.3 from Chrome 131 (Google Security Blog, September 2024); the server side (reverse proxy or Node) decides whether it is used. The client performs no public-key operations of its own; PKCE uses SHA-256 and `crypto.getRandomValues`.

7.6 Mutable runtime tags

`node:lts-alpine` (Dockerfile:15) and `lts/*` (CI) resolve differently over time. Any PQC evaluation result is only valid for the concrete Node/OpenSSL version tested. Recommendation: record the resolved digest in the evaluation report, and decide (Government) whether production should pin a version.

8. Phased PQC migration roadmap

No phase below changes production without a separate, authorized change. Phases 1–4 list what would change *if* authorized, and what to test first.

Phase 0 — Inventory and governance (now)

- Adopt this inventory as the baseline. Re-run the scanner on each release; compare `crypto-inventory.json` between releases and review any new Shor-class row.
- Record deployment-side facts the scanner cannot see: IdP signing algorithm and key size, MySQL server TLS version and cipher, reverse proxy TLS policy, actual Node/OpenSSL version of the deployed image (`node -p process.versions.openssl`), OpenPGP key algorithm behind `STIGMAN_PRIVATE_KEY`.
- Open tracking items for the three external dependencies: IdP ML-DSA support, `jsonwebtoken/jose` ML-DSA support, MySQL server hybrid-group support.
- Decide pinning policy for the runtime image (Decision D5).

Phase 1 — Hygiene (no PQC yet)

- Replace RSA-1024 test fixtures with RSA-3072 or P-256 (`test/utls/mockOidc.js:21,48;` `test/api/mock-keycloak*/.../certs`). Test-only change; still requires authorization because it touches the repository.
- Document and, when authorized, configure an explicit TLS policy: minimum TLS 1.2, prefer TLS 1.3, approved cipher suites (SP 800-52 Rev. 2), HSTS at the reverse proxy. Today none of this is set in the repository (`server.js:45`, absence rows).
- Document that `STIGMAN_OIDC_PROVIDER` must use `https://` in production; the default and the JWKS client permit `http://` (`config.js:95`, `jwksCache.js:92`).
- Document that `STIGMAN_JWT_AUD_VALUE` should be set so audience checking is active (`config.js`, `auth.js:77`).
- Confirm `rejectUnauthorized` behavior for the MySQL connection in a test environment.

Phase 2 — Hybrid key establishment at TLS termination (non-production)

- Target: X25519MLKEM768 (draft-ietf-tls-ecdhe-mlkem) offered by the TLS terminator in front of the API and, where the MySQL server supports it, on the database connection.
- Preferred placement: the reverse proxy or ingress, not the Node process. This leaves application code untouched and centralizes policy.
- Alternative: Node.js `https.createServer` with an explicit `groups/ecdheCurve`-style option on Node 24 + OpenSSL 3.5. This is a code change and is listed only for evaluation.
- Test-and-evaluation plan (non-production):
- Build the evaluation image from a pinned Node 24 digest; record `process.versions.openssl`.
- Terminate TLS at a proxy configured with X25519MLKEM768 first, X25519 fallback. Confirm handshake group with a packet capture or `openssl s_client -groups`.
- Exercise the full API test suite (`test/api`) through the proxy; measure handshake latency and CPU against the classical baseline.
- Confirm browser interoperability with the browsers in the Government's approved list.
- Repeat against a MySQL server built on OpenSSL 3.5 if available; otherwise record the constraint.
- Report: negotiated groups, failures, performance deltas, and any client that fell back to classical.

Phase 3 — Signature migration (when IdP and libraries support ML-DSA)

- Prerequisites: IdP release with ML-DSA signing; JOSE `alg` registration final; a JWT library in the API supporting it; Government approval of ML-DSA parameter set (ML-DSA-65 or ML-DSA-87 per CNSA 2.0 guidance for the system's classification).
- Changes that would be needed (all require authorization): swap or upgrade the JWT library; extend the JWK `kty` filter (`jwksCache.js:188`); optionally pass an explicit `algorithms` allowlist so the accepted set is configuration, not inference.
- Transition pattern: IdP publishes both classical and ML-DSA keys (passive/active rotation). API accepts both during overlap. Cut over when all clients obtain ML-DSA-signed tokens. Then remove classical keys.
- Release signing: generate an ML-DSA (or LMS/XMSS for long-lived, rarely-signed artifacts per SP 800-208) signing key when the signing tool supports it; publish both signatures during overlap; update `root.json` on trust root with the new key type when the registry supports it.
- Evaluation: run the API test suite against the repository's mock OIDC provider modified to sign with ML-DSA; measure token size (ML-DSA-65 signatures are about 3.3 KB) against header limits at the proxy and in the WebSocket upgrade path.

Phase 4 — Validation and retirement of classical-only paths

- Verify every TLS endpoint negotiates a hybrid or PQC group; verify every accepted token is ML-DSA-signed; verify release artifacts carry PQC signatures.
- Remove RSA/EC keys from the IdP JWKS; remove classical alg values from the API allowlist; retire classical-only cipher configuration at the proxy.
- Re-run the scanner; the Shor-class count for non-test scope should be zero except for documented exceptions.
- Timeline anchors: NIST IR 8547 (initial public draft) proposes that 112-bit-security quantum-vulnerable algorithms are deprecated after 2030 and all quantum-vulnerable algorithms are disallowed after 2035; NSM-10 sets 2035 as the national migration goal. The system's own dates must be set by the Government (Decision D7).

9. Risks, interoperability constraints, operational impacts

Risk / constraint	Effect	Mitigation
IdP ML-DSA support arrives late	Phase 3 slips; token forgery exposure persists	Track upstream; keep hybrid TLS (Phase 2) as the near-term control; consider IdP alternatives only with Government approval
jsonwebtoken has no PQC roadmap	Library swap needed	Evaluate jose releases implementing the ML-DSA draft in a test branch; no production change until authorized
Larger tokens and handshakes	ML-DSA signatures and ML-KEM key shares increase header and handshake sizes	Test proxy header limits and WebSocket upgrade; measure latency in Phase 2/3 T&E
Mutable lts tag	Evaluation results not reproducible; behavior changes on 2026-10-28	Pin by digest for evaluation; decide production pinning (D5)
MySQL server lags	Database link stays classical	Accept as documented exception or upgrade server; often on private network
Security level 2 rejects RSA-1024	Any legacy RSA-1024 material fails on Node 24	Replace test fixtures now (Phase 1)
Reverse proxy owns TLS policy	Repository cannot enforce TLS floor or HSTS	Document the required proxy policy as a deployment control and verify in T&E
Hybrid vs. pure PQC	Browsers ship the hybrid group, not a pure ML-KEM group	Recommend hybrid in Phase 2; revisit for Phase 4 (D2)
Test fixtures model RSA only	Tests will not catch ML-DSA regressions	Extend test/utils/mock0idc.js when a library is chosen (Phase 3)

10. Decisions requiring Government approval

- **D1 — Algorithm selection.** Adopt CNSA 2.0 targets as written in the inventory: ML-KEM-768 or ML-KEM-1024 for key establishment; ML-DSA-65 or ML-DSA-87 for signatures; LMS/XMSS for firmware-like release artifacts; SHA-384 for new hash uses. Confirm parameter sets for the system's classification level.

- **D2 — Hybrid versus pure PQC.** Recommend hybrid X25519MLKEM768 for Phase 2 and pure ML-DSA (no composite) for token signatures in Phase 3, with a dual-key overlap. Confirm or direct otherwise.
- **D3 — IdP roadmap dependency.** Accept that Phase 3 is gated on the IdP vendor's ML-DSA release, or direct evaluation of alternatives.
- **D4 — JWT library change.** Authorize a test-branch evaluation of a PQC-capable JOSE library and an explicit algorithms allowlist. No production change without a separate approval.
- **D5 — Runtime pinning.** Pin `node:tls-alpine` and `CI tls/*` to a concrete Node 24 version/digest, or keep the mutable tag and accept re-verification on each move.
- **D6 — TLS policy ownership.** Confirm that TLS version/cipher floor and HSTS are set at the reverse proxy (deployment control), or authorize adding TLS options in the application.
- **D7 — Timelines.** Set target dates for Phases 1–4 against the NSM-10 2035 goal and the NIST IR 8547 draft 2030/2035 transition points.
- **D8 — Test fixture replacement.** Authorize replacing RSA-1024 test keys/certificates with RSA-3072 or P-256 (test-only change).
- **D9 — Release-signing key type.** Confirm the OpenPGP key algorithm behind `STIGMAN_PRIVATE_KEY` and decide when to add a PQC signature.

11. Traceability appendix

11.1 Recommendation → source → evidence

Recommendation	Standard / source	Repository evidence
Baseline inventory, re-run per release	OMB M-23-02 (inventory of cryptographic systems); NSM-10	<code>scripts/crypto_inventory.py</code> ; <code>crypto-inventory.json</code>
Replace RSA-1024 fixtures	SP 800-131A Rev. 2 (RSA < 2048 disallowed); Node.js v22→v24 guide (security level 2)	<code>test/utls/mock0idc.js:21,48</code> ; <code>test/api/mock-keycloak*/.../certs:1</code>
TLS 1.2 floor, TLS 1.3 preferred, approved suites	SP 800-52 Rev. 2; RFC 8446	<code>api/source/bootstrap/server.js:45</code> (no options set); absence rows
HSTS at proxy	SP 800-52 Rev. 2 (server configuration)	absence row anchored at <code>server.js:45</code>
HTTPS-only IdP URL; audience check	RFC 7636 (PKCE) and OIDC practice; SP 800-52 Rev. 2	<code>api/source/utls/config.js:95</code> ; <code>jwtCache.js:92</code> ; <code>auth.js:77</code>
Hybrid X25519MLKEM768 at TLS termination	FIPS 203; draft-ietf-tls-ecdhe-mlkem; CNSA 2.0 (ML-KEM)	<code>server.js:45</code> ; <code>service/utls.js:198</code> ; <code>Dockerfile:15</code>
ML-DSA token signatures when IdP supports it	FIPS 204; draft-ietf-cose-dilithium; CNSA 2.0 (ML-DSA)	<code>auth.js:77</code> ; <code>jwtCache.js:188</code>
ML-DSA or LMS/XMSS for release signing	FIPS 204; FIPS 205; SP 800-208; CNSA 2.0	<code>.github/workflows/build-binary-artifacts.yml:49-82</code> ; <code>root.json:1</code>
Pin runtime for	Node.js release schedule;	<code>Dockerfile:15</code> ; <code>.github/</code>

Recommendation	Standard / source	Repository evidence
evaluation	OpenSSL 3.5 LTS announcement	workflows/api-container-tests.yml:40; api/pkg.config.json:14
2030/2035 transition anchors	NIST IR 8547 ipd; NSM-10	Section 8, Phase 4
SHA-256 acceptable; SHA-384 for new designs	SP 800-131A Rev. 2; CNSA 2.0	STIGService.js:799,802; Attachments.js:141; oidc-worker.js:157,267

11.2 Sources (each URL was opened and confirmed during this assessment)

- NSM-10, National Security Memorandum on Promoting United States Leadership in Quantum Computing While Mitigating Risks to Vulnerable Cryptographic Systems (May 4, 2022): <https://bidenwhitehouse.archives.gov/briefing-room/statements-releases/2022/05/04/national-security-memorandum-on-promoting-united-states-leadership-in-quantum-computing-while-mitigating-risks-to-vulnerable-cryptographic-systems/>
- OMB M-23-02, Migrating to Post-Quantum Cryptography (Nov 18, 2022): <https://www.whitehouse.gov/wp-content/uploads/2022/11/M-23-02-M-Memo-on-Migrating-to-Post-Quantum-Cryptography.pdf>
- NSA Post-Quantum Cybersecurity Resources (CNSA 2.0; points to CNSSP 15, released March 4, 2025): <https://www.nsa.gov/Cybersecurity/Post-Quantum-Cybersecurity-Resources/>
- CNSS Policies index (CNSSP 15): <https://www.cnss.gov/CNSS/issuances/Policies.cfm> (requires a DoD root certificate in the browser)
- FIPS 203, Module-Lattice-Based Key-Encapsulation Mechanism Standard: <https://csrc.nist.gov/pubs/fips/203/final>
- FIPS 204, Module-Lattice-Based Digital Signature Standard: <https://csrc.nist.gov/pubs/fips/204/final>
- FIPS 205, Stateless Hash-Based Digital Signature Standard: <https://csrc.nist.gov/pubs/fips/205/final>
- NIST SP 800-131A Rev. 2, Transitioning the Use of Cryptographic Algorithms and Key Lengths: <https://csrc.nist.gov/pubs/sp/800/131/a/r2/final>
- NIST SP 800-52 Rev. 2, Guidelines for the Selection, Configuration, and Use of TLS Implementations: <https://csrc.nist.gov/pubs/sp/800/52/r2/final>
- NIST SP 800-208, Recommendation for Stateful Hash-Based Signature Schemes: <https://csrc.nist.gov/pubs/sp/800/208/final>
- NIST IR 8547 (initial public draft), Transition to Post-Quantum Cryptography Standards: <https://csrc.nist.gov/pubs/ir/8547/ipd>
- RFC 7518, JSON Web Algorithms: <https://www.rfc-editor.org/rfc/rfc7518>
- RFC 8037, CFRG Elliptic Curve Signatures in JOSE (OKP key type): <https://www.rfc-editor.org/rfc/rfc8037>
- RFC 7636, Proof Key for Code Exchange: <https://www.rfc-editor.org/rfc/rfc7636>
- RFC 8446, TLS 1.3: <https://www.rfc-editor.org/rfc/rfc8446>
- draft-ietf-tls-ecdhe-mlkem (X25519MLKEM768 and related hybrids): <https://datatracker.ietf.org/doc/draft-ietf-tls-ecdhe-mlkem/>
- draft-ietf-cose-dilithium (ML-DSA for JOSE/COSE): <https://datatracker.ietf.org/doc/draft-ietf-cose-dilithium/>
- Node.js 24.5.0 release notes (OpenSSL 3.5 upgrade): <https://nodejs.org/en/blog/release/v24.5.0>
- Node.js v22 to v24 migration guide (OpenSSL 3.5, security level 2): <https://nodejs.org/en/blog/migrations/v22-to-v24>

- Node.js release schedule: <https://github.com/nodejs/Release>
- OpenSSL 3.5 final release (ML-KEM, ML-DSA, SLH-DSA; LTS to April 2030): <https://openssl-library.org/post/2025-04-08-openssl-35-final-release/>
- Google Security Blog, "A new path for Kyber on the web" (Chrome 131 switches to X25519MLKEM768): <https://security.googleblog.com/2024/09/a-new-path-for-kyber-on-web.html>
- Node.js crypto and tls API documentation: <https://nodejs.org/api/crypto.html> and <https://nodejs.org/api/tls.html>
- jsonwebtoken (supported algorithms): <https://github.com/auth0/node-jwebtoken>
- Keycloak Server Administration Guide (signing keys, active/passive rotation): https://www.keycloak.org/docs/latest/server_admin/index.html
- Keycloak pull request for ML-DSA OIDC support (open, unmerged at time of writing): <https://github.com/keycloak/keycloak/pull/50358>

11.3 Inventory row → evidence

Every row in `Cryptographic-Inventory.xlsx` (Inventory sheet) and `crypto-inventory.json` carries `file`, `line`, `rule` and a redacted evidence string. Rows that record the absence of a control (ABSENT-*) are anchored to the TLS server setup line and state the pattern that was not found.